Ticket: HDFC Bank Wallet Types

Customer: HDFC Bank campus wallet desk
Need: One **Account** blueprint that cannot be constructed. Savings and wallet **IS-A** account. Same loop prints both. Wallet can **pay** through a `Payable` contract.

Teaching-order: `abstract`, `implements`, polymorphism (`Account[]`). Arrays already taught — **no** **`ArrayList`**. Validate with `print` + `return` — **no** **`throw`** **/** **`try`** **/** **`catch`**. No Spring. No Lombok.

Starter

Complete `kind()`, `SavingsAccount`, `WalletAccount`, and `Payable.pay` in `starter/CampusWallet.java`. Keep `balance` private on the abstract parent.

Broken

`Account` is concrete (`new Account(...)` works). `kind()` always returns `"account"`. Wallet has a misspelled `paay` — not the interface method. Checkout uses `if (type == ...)`.

Done when

- [ ] `Account` is `abstract` — `new Account(...)` does not compile
- [ ] `SavingsAccount` and `WalletAccount` extend `Account` and `@Override kind()`
- [ ] `WalletAccount implements Payable` and `pay` uses `withdraw` (print + return)
- [ ] `Account[]` loop prints both kinds without `if (a instanceof ...)`
- [ ] `balance` stays private; no `setBalance`
- [ ] No `throw` / `try` / `catch`
- [ ] No `ArrayList`
- [ ] No Lombok

Stretch

- Tiny CLI menu (Deposit / Withdraw / Pay / Interest / List / Quit) around the array.
- `CurrentAccount extends Account` with a different withdraw rule (still print + return).
- **Later — Collections week:** `ArrayList<Account>` instead of a fixed array.
- **Later — Exceptions week:** `throw` on bad amounts; catch in the menu.